Week-6

Aim: Implement distance vector routing algorithm for obtaining routing tables at each node.

Source Code:

```
#include <stdio.h>
struct node
{
   unsigned dist[20];   // Stores minimum distance to each destination
   unsigned from[20];  // Stores the next hop/router used to reach destination
} rt[10];
int main()
{
   int dmat[20][20];
   int n, i, j, k;
   int count;
   // Read number of routers/nodes
   printf("Enter the number of nodes: ");
   scanf("%d", &n);
   // Read the cost matrix
   printf("Enter the cost matrix:\n");
   for (i = 0; i < n; i++)
   {
      for (j = 0; j < n; j++)
      {
         scanf("%d", &dmat[i][j]);
         // Distance from a node to itself is always 0
         dmat[i][i] = 0;
         // Initially, distance is the direct link cost
         rt[i].dist[j] = dmat[i][j];
         // Initially, destination itself is considered as next hop
         rt[i].from[j] = j;
      }
   }
```

```
/*
   Distance Vector calculation
   Each router checks whether going through
   another router gives a smaller distance.
*/
do
{
    count = 0;
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            for (k = 0; k < n; k++)
            {
                /*
                   Check whether the path:
                   i -> k -> j
                   is cheaper than the current path:
                   i -> j
                */
                if (rt[i].dist[j] >
                    dmat[i][k] + rt[k].dist[j])
                {
                    // Update the shortest distance
                    rt[i].dist[j] =
                        dmat[i][k] + rt[k].dist[j];
                    // Router k becomes the next hop
                    rt[i].from[j] = k;
                    // A change has occurred
                    count++;
                }
            }
```

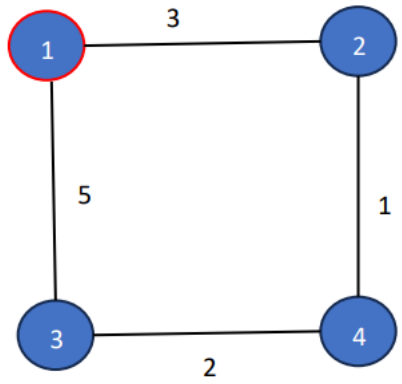
```
            }
        }
    } while (count != 0);
    /*
      Display the final routing tables
    */
    for (i = 0; i < n; i++)
    {
        printf("\n\nRouting table for Router %d\n", i + 1);

        printf("Destination\tNext Hop\tDistance\n");

        for (j = 0; j < n; j++)
        {
            printf("%d\t\t%d\t\t%d\n",
                   j + 1,
                   rt[i].from[j] + 1,
                   rt[i].dist[j]);
        }
    }

    return 0;
}
```

Output: Let the graph be:




```
Enter the number of nodes: 4
Enter the cost matrix:
0 3 5 999
3 0 999 1
5 999 0 2
999 1 2 0


Routing table for Router 1
Destination     Next Hop        Distance
1               1               0
2               2               3
3               3               5
4               2               4


Routing table for Router 2
Destination     Next Hop        Distance
1               1               3
2               2               0
3               4               3
4               4               1

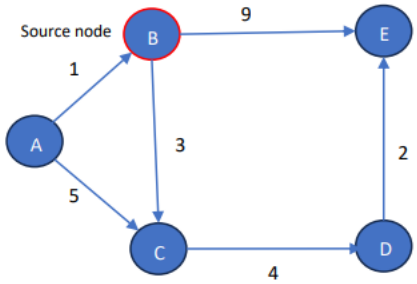

Routing table for Router 3
Destination     Next Hop        Distance
1               1               5
2               4               3
3               3               0
4               4               2


Routing table for Router 4
Destination     Next Hop        Distance
1               2               4
2               2               1
3               3               2
4               4               0
```

Theory:

A distance-vector routing (DVR) protocol requires that a router inform its neighbors of topology changes periodically. Historically known as the old ARPANET routing algorithm (or known as Bellman-Ford algorithm). Bellman Ford Basics – Each router maintains a Distance Vector table containing the distance between itself and ALL possible destination nodes. Distances,based on a chosen metric, are computed using information from the neighbors' distance vectors. Information kept by DV router - • Each router has an ID • Associated with each link connected to a router, • there is a link cost (static or dynamic). • Intermediate hops Distance Vector Table Initialization - • Distance to itself = 0 • Distance to ALL other routers = infinity number. Distance Vector Algorithm – 1. A router transmits its distance vector to each of its neighbors in a routing packet. 2. Each router receives and saves the most recently received distance vector from each of its neighbors. 3. A router recalculates its distance vector when: • It receives a distance vector from a neighbor containing different information than before. • It discovers that a link to a neighbor has gone down. The DV calculation is based on minimizing the cost to each destination Dx(y) = Estimate of least cost from x to y C(x,v) = Node x knows cost to each neighbor v Dx = [Dx(y): y ? N] = Node x maintains distance vector Node x also maintains its neighbors' distance vectors – For each neighbor v, x maintains Dv = [Dv(y): y ? N]

Example: Distance vector routing table for Hop B.



Destination	Cost	Next Hop
A	1	A (directed connected to B)
C	3	C (directed connected to B)
E	9	E (directed connected to B)
D	7	C (intermediate node to D)

B to A

Destination	Intermediate Node	Next Hop	Cost
B	**-**	**A**	**1 (direct connected with intermediate node)**
B	C	A	8
B	E-D	C	20

B to C

Destination	Intermediate Node	Next Hop	Cost
B	**-**	**C**	**3(direct connected with intermediate node)**
B	E-D	C	15
B	A	C	06

B to E

Destination	Intermediate Node	Next Hop	Cost
B	**-**	**E**	**9 (direct connected with intermediate node)**

B	C-D	E	9(even value same since its connected by intermediate node not considered)
B	A-C-D	E	12

B to D

Destination	Intermediate Node	Next Hop	Cost
B	E	D	11
B	**C**	**D**	**7(viva intermediate node with Minium cost is considered)**
B	A-C	D	10